

LIV-TS: Evolutionary Architecture Search for Time Series Forecasting via Linear Input-Varying Systems

Anonymous Authors

Abstract—Contemporary state-of-the-art time series forecasting models—including DMamba, S-Mamba, and iTransformer—achieve strong performance but hardcode specific operator choices (Mamba recurrence, self-attention, or feedforward networks) for each architectural component. This rigid assignment assumes a universal inductive bias that may not hold across diverse datasets and forecasting horizons. We propose LIV-TS, a framework that casts time series forecasting architecture design as a multi-objective evolutionary search over *Linear Input-Varying* (LIV) operators. LIV provides a unified parameterization of attention, state-space models (SSMs), gated convolutions, and feedforward networks under a single token-mixing matrix whose structure determines the operator family. We instantiate LIV operators inside two established forecasting structures—S-Mamba and iTransformer—and apply NSGA-II to jointly optimize forecasting accuracy and model compactness. Additionally, we adapt the Think-in-Diffusion Autoregressive (TiDAR) training scheme for continuous-valued time series, enabling parallel draft prediction with optional autoregressive refinement. Experiments on the ETT benchmark and Exchange-Rate dataset reveal that evolutionary search consistently discovers *heterogeneous* operator mixtures that outperform any single fixed operator class. The NSGA-II best candidate for S-Mamba on ETTh1 (H=96) achieves a test MSE of 0.596, with only 528K parameters—a $32\times$ reduction over the all-SSM baseline (17M parameters) without sacrificing accuracy. The TiDAR draft mode delivers an $80.8\times$ inference speedup over full autoregressive decoding with negligible accuracy loss. Cross-dataset architecture transfer experiments show that genomes discovered on ETTh1 generalize to ETTh2 and Exchange-Rate, reducing MSE by up to 63% relative to the all-SSM baseline. Our results demonstrate that operator choice is a critical yet underexplored axis of model design in time series forecasting, and that evolutionary search over a unified LIV operator pool reliably identifies compact, high-quality architectures.

Index Terms—Time series forecasting, neural architecture search, linear input-varying systems, state-space models, evolutionary optimization, NSGA-II, attention mechanisms, autoregressive refinement.

I. INTRODUCTION

Multivariate time series forecasting is a cornerstone task in domains ranging from energy grid management and financial trading to climate science and traffic control [1], [2]. Accurate multi-step predictions over long horizons require models that can simultaneously capture local temporal patterns, long-range dependencies, and inter-variate correlations—three qualitatively different phenomena that may benefit from different computational inductive biases.

Recent architectures have tackled this challenge by specializing each component: iTransformer [3] treats each variate as

a token and uses self-attention for cross-variate correlation alongside a feedforward network (FFN) for per-variate feature extraction; S-Mamba [4] applies bidirectional Mamba recurrence for variate correlation followed by an FFN for temporal refinement. These designs encode strong prior beliefs: *attention is best for cross-variate mixing* and *Mamba recurrence is best for temporal modeling*. While each assumption yields strong results on particular benchmarks, it is unclear whether any single operator family universally excels, or whether the optimal operator varies with dataset characteristics and forecast horizon.

A. Motivation: Why Fixed Operator Choices May Be Suboptimal

The core tension is illustrated by comparing operator families under a controlled benchmark. On ETTh1 at horizon $H=96$ using the iTransformer structure, replacing self-attention with 3-tap gated convolution (GConv-1) reduces test MSE from 0.476 to 0.467 while using 33% fewer parameters. Conversely, using Mamba SSM (Rec-1) across all slots inflates parameter count to 17M and degrades MSE to 1.213. These results suggest that the *choice of operator family per slot* is a high-impact yet underexplored design dimension.

Manual search over all combinations is intractable: with eleven base operator classes, four slots per structure, and differential variants, the search space exceeds 10^5 configurations per structure. Automated neural architecture search (NAS) [5], [6] offers a principled alternative, but existing NAS methods for time series focus on hyperparameter tuning rather than operator-family selection.

B. Our Approach: LIV-TS

We propose **LIV-TS**, which addresses the operator-selection problem through two components:

1. Linear Input-Varying (LIV) unification. Following the LIV framework [14], [15], we express all mixing operators—self-attention variants, state-space models, gated convolutions, and feedforward networks—as instances of a token-mixing matrix $T_{ij}(x)$ whose structural pattern determines the operator family:

- **LOW_RANK**: $T = CB^\top/\sqrt{d}$ — recovers multi-head self-attention (SA-1 through SA-4);
- **SEMI_SEPARABLE**: $T_{ij} = C_i \prod_{k=j}^{i-1} A_k B_j$ — recovers SSMs including Mamba (Rec-1/2) and discretized S4

(Rec-3), and Closed-form Continuous-time networks / CfC (Rec-4);

- **TOEPLITZ**: $T_{ij} = C_i K_{i-j} B_j$ — recovers local (GConv-1) and long-range input-dependent convolutions (GConv-2);
- **DIAGONAL**: $T_{ij} = C_i B_i \delta_{ij}$ — recovers element-wise gating (SwiGLU FFN).

This unification lets us treat all operators as interchangeable genome entries.

2. NSGA-II evolutionary search. We encode each forecasting architecture as a genome of LIV class identifiers—one per operator slot—and run NSGA-II [8] to simultaneously minimize validation MSE and parameter count. The Pareto frontier balances accuracy against model compactness without requiring explicit weighting.

Beyond architecture search, we integrate the Think-in-Diffusion Autoregressive (TiDAR) training scheme [9] into LIV-TS, enabling parallel draft prediction of all H future steps in a single forward pass with optional autoregressive refinement steps. This decoupling of *quality* (full AR) from *speed* (parallel draft) is particularly valuable for long-horizon settings ($H=720$) where autoregressive decoding is prohibitively slow.

C. Contributions

The main contributions of this work are:

- 1) **LIV-TS framework.** We formalize time series forecasting architecture search within the LIV operator algebra, providing a clean genome encoding compatible with two structural templates: S-Mamba-style and iTransformer-style.
- 2) **Multi-objective evolutionary search.** We demonstrate that NSGA-II discovers heterogeneous operator mixtures—combining convolution, attention, and SSM variants within a single model—that dominate any single-class baseline on the Pareto frontier of accuracy versus parameter count. On ETTh1 $H=96$, the best evolved S-Mamba genome achieves test MSE=0.596 with only 528K parameters, compared to 17M parameters for the all-SSM baseline (MSE=0.999).
- 3) **TiDAR adaptation for continuous time series.** We adapt TiDAR’s joint autoregressive–diffusion loss to continuous-valued forecasting and show that the draft mode achieves an $80.8\times$ wall-clock speedup over full autoregressive decoding with test MSE degradation of only 0.004 absolute.
- 4) **Cross-dataset architecture transfer.** We show that genomes discovered by evolutionary search on ETTh1 transfer non-trivially to ETTh2 and Exchange-Rate, outperforming the all-SSM baseline by 32%–63% in test MSE without retraining the genome, providing evidence that evolutionary search recovers structurally meaningful inductive biases.
- 5) **CfC operator for time series.** We introduce the Closed-form Continuous-time (CfC) recurrence [10] as an LIV operator class (Rec-4) and show it achieves comparable accuracy to Mamba (Rec-1) on ETTh1 while offering a complementary gating mechanism suited to non-stationary continuous dynamics.

D. Paper Organization

Section II reviews the LIV operator algebra, existing forecasting structures, and the NSGA-II algorithm. Section III describes the LIV-TS genome encoding, structural templates, fitness evaluation, and TiDAR integration. Section IV presents experiments on operator ablation, full architecture search, TiDAR speedup, and cross-dataset transfer. Section ?? analyzes when heterogeneous architectures outperform single-operator models and characterizes the discovered Pareto fronts. Section ?? concludes and outlines future directions.

II. BACKGROUND AND RELATED WORK

A. Time Series Forecasting Architectures

Multivariate long-sequence forecasting has evolved through several architectural paradigms. Early Transformer-based models such as Informer [2] and Autoformer [1] apply attention over the *temporal* axis, treating each time-step as a token. This yields $\mathcal{O}(L^2)$ complexity and conflates temporal modelling with cross-variate modelling, since each token mixes all C variates at a single time-step.

A decisive shift came with the observation that *inverting* the token definition—making each variate a token—decouples variate correlation from temporal structure and reduces attention cost to $\mathcal{O}(C^2)$ when $C \ll L$. iTransformer [3] operationalises this by embedding each variate’s full L -step history into a single D -dimensional token, applying self-attention across the C variate tokens for cross-variate mixing, and a per-variate FFN for feature transformation. S-Mamba [4] adopts the same variate-token layout but replaces the single backbone with a two-stage pipeline: a bidirectional Mamba block for variate correlation followed by an FFN block for temporal refinement. DLinear [11] challenges the necessity of complex operators entirely, showing that a simple decomposition plus linear projection is surprisingly competitive, motivating investigation into whether operator complexity should be matched to each task. PatchTST [12] and TimesNet [13] explore patched temporal tokens and 2D convolutional representations respectively, further diversifying the design space.

Across these models a common pattern emerges: each design *commits* to a specific operator for each architectural slot without empirical justification for that choice. This motivates an operator-agnostic search over a unified operator family.

B. Linear Input-Varying (LIV) Operators

The LIV framework [14], [15] provides a unified algebraic view of sequence mixing operators. Its central insight is to replace the *fixed* weight matrix of a standard linear layer with a weight that is *assembled from the input itself*.

In a standard linear layer the output is $Y = Wx$, where W is a learned parameter independent of x . In a LIV block, four learned linear projections of x first produce feature vectors B , C , S , V :

$$B = W_B x, \quad C = W_C x, \quad S = f_S(x), \quad V = W_V x, \quad (1)$$

where W_B, W_C, W_V are plain `nn.Linear` layers (single matrix multiplies) and f_S is either a linear layer with a learned

bias or, for the input-dependent convolution case, a small two-layer MLP. These are called the *featurizer*.

The $L \times L$ token-mixing matrix T is then *assembled algebraically* from B , C , and S — the four projections have distinct roles:

- $\mathbf{b}_j$ and $\mathbf{c}_i$ (rows of B and C) **dot-product to produce the scalar gate strength** of each entry T_{ij} . Their inner product $\langle \mathbf{c}_i, \mathbf{b}_j \rangle$ measures how strongly position i should attend to position j .
- S **provides the structural scalar** M_{ij} that further modulates the gate strength: it encodes the *pattern* (a decay factor for SSMs, a kernel value for convolutions, the constant 1 for attention, or absent for the diagonal case).
- V **carries the payload**: it never enters the construction of T at all. Once T is assembled, the token-mixed output is obtained by $T \cdot V$, which routes the actual feature content.

Concretely, each entry of T is:

$$T_{ij} = \underbrace{\langle \mathbf{c}_i, \mathbf{s}_{ij} \odot \mathbf{b}_j \rangle}_{\text{gate strength modulated by structure}}, \quad (2)$$

where $\mathbf{s}_{ij} \in \mathbb{R}^d$ is the *structural modulator* derived from S — for LOW_RANK it is the all-ones vector; for SEMI_SEPARABLE it is the element-wise cumulative product $\prod_{k=j}^{i-1} \mathbf{a}_k$ of transition vectors from S ; for TOEPLITZ it is a scalar K_{i-j} broadcast from S .

The full LIV forward pass is:

$$\text{mixed}_i = \sum_j T_{ij} \cdot \mathbf{v}_j, \quad \mathbf{v}_j = j\text{-th row of } V, \quad (3)$$

$$Y_i = W \cdot \text{mixed}_i, \quad (4)$$

where W is a *static* nn.Parameter — a fixed learned matrix independent of x . The net computation is: (1) project x to B, C, S, V via linear layers; (2) assemble T from B, C, S algebraically; (3) mix content with $\text{mixed} = T \cdot V$; (4) project with fixed W .

1) *Token Mixing Families*: Four choices of how S constructs $\mathbf{s}_{ij}$ span the space of practical sequence operators.

LOW_RANK — $\mathbf{s}_{ij} = 1/\sqrt{d}$ (constant, S unused):

$$T_{ij} = \langle \mathbf{c}_i, \mathbf{b}_j \rangle / \sqrt{d}, \quad T = CB^\top / \sqrt{d}. \quad (5)$$

Every pair (i, j) interacts with equal structural weight; row-wise softmax recovers multi-head attention [16]. Cost $\mathcal{O}(L^2d)$; full bidirectional context.

SEMI_SEPARABLE — $\mathbf{s}_{ij} = \prod_{k=j}^{i-1} \mathbf{a}_k$ (S gives forget gate $\mathbf{a}_k$ per step):

$$T_{ij} = \left\langle \mathbf{c}_i, \left(\prod_{k=j}^{i-1} \mathbf{a}_k \right) \odot \mathbf{b}_j \right\rangle \quad (j \leq i), \quad T_{ij} = 0 \quad (j > i), \quad (6)$$

where $\mathbf{a}_k \in (0, 1)^d$ is the per-step forget gate (the k -th row of S), and $\mathbf{b}_j$ is the input contribution at position j . The structural product $\prod_{k=j}^{i-1} \mathbf{a}_k$ decays the contribution of position j exponentially as it propagates forward to position i ; it is computed once via `torch.cumprod` and shared across all pairs without any per-element network call. In this work we use CfC [17] as the sole SEMI_SEPARABLE operator; its

complementary gating specialises the featurizer as described in Section II-D below.

TOEPLITZ — $\mathbf{s}_{ij} = K_{i-j} \mathbf{1}$ (S provides scalar kernel, zero beyond window):

$$T_{ij} = \langle \mathbf{c}_i, \mathbf{b}_j \rangle \cdot K_{i-j}, \quad K_{i-j} = 0 \text{ if } |i-j| \geq k. \quad (7)$$

S is averaged across the feature dimension to yield a scalar kernel vector $K \in \mathbb{R}^k$ indexed by offset. For a fixed kernel K is an nn.Parameter; for the input-dependent variant, a two-layer MLP maps the global average of x to K — this is the only operator where $f_S(x)$ involves a nonlinear network [7]. Cost $\mathcal{O}(Lkd)$ for window size k .

DIAGONAL — $\mathbf{s}_{ij} = 1$ only when $i = j$, else zero (S unused):

$$T_{ij} = \langle \mathbf{c}_i, \mathbf{b}_i \rangle \delta_{ij}, \quad \text{mixed}_i = \langle \mathbf{c}_i, \mathbf{b}_i \rangle \mathbf{v}_i. \quad (8)$$

Only the diagonal of T is non-zero; the scalar $\langle \mathbf{c}_i, \mathbf{b}_i \rangle$ gates how strongly position i 's value $\mathbf{v}_i$ passes through. With $\mathbf{c}_i = \mathbf{1}$ and $\mathbf{b}_i = \text{SiLU}(W_{\text{gate}} x_i)$, this recovers the SwiGLU FFN [18] at $\mathcal{O}(Ld)$ cost with no cross-token interaction.

The four families trade off context range against cost:

$$\text{DIAGONAL} < \text{TOEPLITZ} < \text{SEMI_SEPARABLE} \approx \text{LOW_RANK} \quad (\text{in context width})$$

while SEMI_SEPARABLE and DIAGONAL share $\mathcal{O}(Ld)$ recurrent complexity versus $\mathcal{O}(L^2d)$ for LOW_RANK.

C. Recurrent Sequence Models

Structured State-Space models (S4 [19]) and Mamba [20] parameterise causal recurrences where a hidden state h_t summarises all past inputs. Both belong to the SEMI_SEPARABLE family in the LIV taxonomy: S supplies the per-step transition scalar A_k and the T matrix unrolls the recurrence as a lower-triangular matrix of decayed contributions. They are included here as the direct conceptual predecessors of CfC, which is the SEMI_SEPARABLE operator used in this work.

D. CfC-LNN and its LIV Representation

CfC (Closed-form Continuous-time) networks [17], derived from Liquid Time-Constant neurons [10], solve continuous-time recurrent dynamics in closed form. The standard CfC state update is:

$$h_t = g_t \odot h_{t-1} + (1 - g_t) \odot \tilde{h}_t, \quad g_t = \sigma(W_g x_t + b_g), \quad (9)$$

where $g_t \in (0, 1)^d$ is the *forget gate* and $(1 - g_t)$ is the *input gate*, with the complementary constraint $g_t + (1 - g_t) = 1$ enforcing a convex interpolation between the retained state and the new input $\tilde{h}_t = \tanh(W_{\text{in}} x_t)$.

1) *CfC as a SEMI_SEPARABLE LIV Block*: Unrolling Eq. (9) from position j to position i shows that h_i is a weighted sum of all past inputs:

$$h_i = \sum_{j=0}^i \underbrace{\left(\prod_{k=j+1}^i g_k \right)}_{\text{survival from } j \text{ to } i} \odot \underbrace{(1 - g_j) \odot \tilde{h}_j}_{\text{input absorbed at } j}. \quad (10)$$

This is exactly the SEMI_SEPARABLE token mixing. The LIV featurizer for CfC maps each input token to:

$$\mathbf{b}_j = (1 - g_j) \odot W_{\text{in}} x_j, \quad \mathbf{a}_k = g_k = \sigma(W_g x_k), \quad \mathbf{c}_i = W_C x_i, \quad (11)$$

so each entry of $T(x)$ becomes:

$$T_{ij} = \left\langle W_C x_i, \left(\prod_{k=j}^{i-1} \sigma(W_g x_k) \right) \odot (1 - \sigma(W_g x_j)) \odot W_{\text{in}} x_j \right\rangle \quad (12)$$

The scalar T_{ij} thus encodes: *how much of position j 's contribution*, modulated by its own input gate $(1 - g_j)$, *survives* through the chain of forget gates $g_j \cdot \dots \cdot g_{i-1}$, *as read out* by position i 's output projection $\mathbf{c}_i$. The token-mixed output $\text{mixed}_i = \sum_{j \leq i} T_{ij} \mathbf{v}_j$ is a weighted sum of value vectors, where the weight of each past position decays with the cumulative product of all intermediate forget gates — recovering precisely the CfC hidden state h_i in Eq. (10).

The complementary constraint $g_t + (1 - g_t) = 1$ ensures that each gate pair (A_k, B_k) partitions unity, stabilising gradient flow through the cumulative product and making CfC particularly well-suited to smooth continuous-valued signals such as exchange rates and temperature series.

E. NSGA-II

NSGA-II [8] is a fast elitist evolutionary algorithm for multi-objective optimisation. It maintains a population ranked by Pareto dominance: an individual a dominates b if a is no worse than b on all objectives and strictly better on at least one. Rank-0 individuals form the Pareto front. Within each rank, crowding distance prioritises individuals in sparse objective-space regions, preserving diversity. Parents are selected by tournament on (rank, crowding distance); offspring are produced by multi-point crossover and per-gene mutation. Elitism preserves the top individuals across generations, guaranteeing monotone improvement of the Pareto front.

F. TiDAR: Think-in-Diffusion Autoregressive Prediction

Standard autoregressive (AR) forecasting generates the horizon sequentially at $\mathcal{O}(H)$ forward passes. TiDAR [9] proposes a hybrid training objective that enables parallel *draft* prediction of all H steps in a single pass, with optional AR refinement.

The $L + H$ tokens are processed jointly under a hybrid sparsity mask: lookback tokens attend causally among themselves; forecast positions are masked but attend bidirectionally to the full lookback. The training loss combines a next-step AR objective on the lookback with a parallel prediction objective on the masked horizon:

$$\mathcal{L}_{\text{TiDAR}} = \frac{1}{1 + \alpha} (\alpha \cdot \mathcal{L}_{\text{AR}} + \mathcal{L}_{\text{Diff}}). \quad (13)$$

At inference, three operating modes trade quality against speed: pure draft ($\mathcal{O}(1)$), partial-AR ($\mathcal{O}(k)$), and full-AR ($\mathcal{O}(H)$).

G. Related Work

NAS for time series. Existing automated search for time series focuses on hyperparameter optimisation (learning rate, hidden size, patch length) rather than operator-family selection [5]. Cell-based NAS [6] has been applied to RNN design but not to the modern SSM/attention/convolution pool.

Unified sequence operators. Mamba [20] reframes SSMs as selective state spaces; Hyena [7] generalises convolution to long implicit kernels. The LIV formalism [14], [15] subsumes these under a single token-mixing abstraction, making it a natural grammar for operator search.

Parallel sequence decoding. Non-autoregressive Transformers [21] and diffusion language models demonstrate that parallel generation is viable in discrete settings. TiDAR [9] extends this to a hybrid draft-then-refine framework suitable for continuous signals.

III. LIV-TS METHODOLOGY

LIV-TS has three components: (1) an *operator pool* of LIV classes that form the search alphabet; (2) two *structural templates* that specify how operator slots are arranged within a forecasting model; and (3) a *search and training protocol* combining NSGA-II evolution with TiDAR-TS parallel training. Figure ?? illustrates the overall pipeline.

A. LIV-TS Operator Pool

We select eleven base operator classes spanning all four LIV token-mixing families. Each class pairs a featurizer (how B , C , V are computed from x) with a token-mix type (the structure of M_{ij}) and a channel-mix type (the structure of W). Table I lists the full pool.

1) *Attention Family (LOW_RANK)*: All attention variants use $T = CB^T / \sqrt{d}$ with softmax normalisation. They differ in how the key (B), query (C), and value (V) projections are computed, trading parameter count against expressiveness.

SA-1 — Standard Multi-Head Attention. All heads use independent key, query, and value projections over the full dimension d . Channel mixing is GROUPED (one projection per head).

SA-2 — Convolution-Pre MHA. A causal depthwise Conv1d (kernel size 3) is applied to x before all projections, injecting local context into each attention head without additional parameters.

SA-3 — Multi-Query Attention [22]. A single shared key-value head is broadcast to all h query heads, reducing KV parameter count to $1/h$ of MHA at the cost of representational capacity.

SA-4 — Grouped-Query Attention [23]. $h/2$ KV head groups, each serving two query heads. This interpolates between MHA (full expressiveness) and MQA (maximal compression).

2) *Recurrence Family (SEMI_SEPARABLE)*: We use a single SEMI_SEPARABLE operator: **Rec-4 (CfC)**, the Closed-form Continuous-time network [17]. SSM variants such as Mamba [20] and S4 [19] also map to SEMI_SEPARABLE and are included in the pool as ablation baselines (Section IV-C),

TABLE I: LIV-TS operator pool. Expansion refers to the ratio of internal dimension to model dimension d .

ID	Name	Token Mix	Channel Mix
1	SA-1 (MHA)	LOW_RANK	GROUPED
2	SA-2 (Conv-MHA)	LOW_RANK	GROUPED
3	SA-3 (MQA)	LOW_RANK	GROUPED
4	SA-4 (GQA)	LOW_RANK	GROUPED
5	Rec-4 (CfC)	SEMI_SEPARABLE	DIAGONAL
6	GConv-1	TOEPLITZ	DIAGONAL
7	GConv-2	TOEPLITZ	DIAGONAL
8	GMemless (FFN)	DIAGONAL	DENSE

Classes 9–16 are differential variants of the above (compute $\text{LIV}_1(x)$ the search space from $|\text{pool}|^N \times 3^N$ to $|\text{pool}|^N$.

but CfC is the sole recurrence operator carried through to the main search and transfer experiments.

Rec-4 — CfC. The featurizer computes complementary forget and input gates from each token:

$$g_t = \sigma(W_g x_t + b_g), \quad \mathbf{b}_t = (1 - g_t) \odot W_{\text{in}} x_t, \quad \mathbf{a}_t = g_t, \quad (14)$$

with internal expansion $16\times$ and DIAGONAL channel mixing. The constraint $g_t + (1 - g_t) = 1$ is satisfied by construction; no additional learnable bias $a_{\log}$ is required. As shown in Section II-D, substituting Eq. (14) into the SEMI_SEPARABLE T_{ij} formula recovers precisely the unrolled CfC recurrence.

3) *Convolution Family (TOEPLITZ):* Both convolution variants use $T_{ij} = C_i K_{i-j} B_j$ with sigmoid gates for B and C . Channel mixing is DIAGONAL. The value projection is the raw input ($V = x$), keeping parameter count low.

GConv-1 — Short Gated Convolution. The kernel $K \in \mathbb{R}^{d \times 3}$ is a fixed learned parameter shared across all inputs. The 3-tap window captures immediate neighbourhood structure at minimal cost.

GConv-2 — Long Input-Dependent Convolution. Following the Hyena implicit-kernel approach [7], a two-layer MLP generates a 64-tap kernel from the global average of the input:

$$K = \text{MLP}(\text{mean}(x)) \in \mathbb{R}^{d \times 64}. \quad (15)$$

The kernel depends on the current sequence, providing global context awareness at $\mathcal{O}(Ld)$ cost.

4) *Memoryless Family (DIAGONAL):* **GMemless — SwiGLU FFN.** Element-wise gating with $4\times$ internal expansion:

$$\text{out}_i = \text{SiLU}(W_{\text{gate}} x_i) \odot (W_{\text{val}} x_i). \quad (16)$$

Channel mixing is DENSE (full output projection). No cross-token interaction; each position is processed independently.

5) *Differential Variants:* Every base class $c \in [1, 9]$ has a *differential* counterpart $c + 9 \in [10, 18]$, making a pool of 18 operator classes in total (plus Rec-3 and Rec-4 differentials at classes 19–21 for the extended pool). A differential operator instantiates *two* independent featurizers with the same class and computes their difference before channel mixing:

$$\text{mixed}_i^{\text{diff}} = \text{LIV}_1(x)_i - \text{LIV}_2(x)_i. \quad (17)$$

This doubles expressiveness at approximately $2\times$ the parameter cost and can be interpreted as a learned contrast between two distinct views of the input.

B. Genome Encoding

A genome g is a fixed-length list of integer operator class IDs, one per slot in the model. Each slot corresponds to one LIV block sub-layer. With N total blocks the genome length is $N\times$ for iTransformer and N for S-Mamba (both structures have one active LIV class per block, but the slot *role* differs by structure as described below).

The channel-mix type is not searched independently: it is derived deterministically from the token-mix family of the selected class, following the pairing in Table I. This reduces the search space from $|\text{pool}|^N \times 3^N$ to $|\text{pool}|^N$.

C. Structural Instantiation

1) *S-Mamba Instantiation:* The S-Mamba structure embeds each variate’s L -step history as a D -dimensional token and processes C variate tokens through two sequential backbones:

$$(B, L, C) \rightarrow (B, C, L) \xrightarrow{\text{Linear}(L \rightarrow D)} (B, C, D). \quad (18)$$

The genome of length N is split in half. **Layers** $0 \dots N/2 - 1$ drive the *variate backbone* (Stage 1): each LIV block mixes information across the C variate tokens bidirectionally, building correlated representations. **Layers** $N/2 \dots N - 1$ drive the *temporal backbone* (Stage 2): each LIV block refines temporal structure within the D -dimensional variate embeddings. A linear head $D \rightarrow H$ per variate produces the forecast, transposed to (B, H, C) .

The original S-Mamba [4] is recovered by setting all Stage-1 slots to Rec-1 (Mamba) and all Stage-2 slots to GMemless (FFN).

2) *iTransformer Instantiation:* The iTransformer structure uses a single backbone of N stacked blocks. Each block contains two sub-layers that alternate roles: the *cross-variate* sub-layer mixes information across the C variate tokens, and the *per-variate* sub-layer acts independently on each variate’s D -dimensional representation.

Within a block of index b , genome entry $2b$ specifies the cross-variate operator and entry $2b + 1$ specifies the per-variate operator (for a total genome length $2N$). The original iTransformer [3] is recovered by setting all cross-variate slots to SA-1 and all per-variate slots to GMemless.

Both structures share the same input/output interface $(B, L, C) \rightarrow (B, H, C)$ and use Reversible Instance Normalisation (RevIN [24]) to remove per-variate distribution shift before the backbone and restore it after.

D. TiDAR-TS Adaptation

We adapt TiDAR [9] to continuous-valued regression. The original TiDAR targets discrete language tokens with cross-entropy loss. Our adaptation makes three changes:

(1) **MSE regression loss.** Both the AR and draft objectives use mean squared error on un-normalised (RevIN-restored) outputs:

$$\mathcal{L}_{\text{AR}} = \text{MSE}(\hat{x}_{2:L}, x_{2:L}), \quad \mathcal{L}_{\text{Diff}} = \text{MSE}(\hat{y}_{1:H}, y_{1:H}). \quad (19)$$

(2) **Deterministic draft.** The draft head directly predicts the forecast mean; there is no stochastic denoising or sampling

TABLE II: NSGA-II search configuration.

Hyperparameter	Value
Population size	16
Generations	18
Evaluation steps / genome	500
Mutation probability	0.10
Elitism (carry-over)	2
Crossover points	multi-point on layer list
Post-search full training	20,000 steps (top- K Pareto front)

step. Draft outputs are used as-is or overwritten by autoregressive refinement for the first k positions.

(3) α as a training hyperparameter. The balance coefficient α in Eq. (13) controls how strongly the AR auxiliary task regularises the backbone. We treat α as a tunable hyperparameter and ablate values $\alpha \in \{0, 0.1, 0.5, 1.0, 2.0, 5.0\}$ in Section IV-G. $\alpha = 0$ recovers pure parallel prediction; $\alpha \rightarrow \infty$ recovers a standard AR model with an auxiliary draft head.

E. NSGA-II Search Protocol

1) *Objectives:* We define two minimisation objectives for each genome g :

$$f_1(g) = \mathcal{L}_{\text{val}}(g) \quad (\text{validation MSE after } N_{\text{eval}} \text{ steps}), \quad (20)$$

$$f_2(g) = \Phi(g) \quad (\text{total parameter count}). \quad (21)$$

No weighting is applied; NSGA-II tracks the Pareto front of accuracy versus compactness automatically.

2) *Search Configuration:* Table II lists the NSGA-II hyperparameters used in all experiments.

Each candidate genome is trained with the Adam optimiser for $N_{\text{eval}} = 500$ steps on a single dataset/horizon pair; validation MSE and parameter count are recorded. After all 18 generations, the top- K Pareto-front genomes (ranked by validation MSE) are retrained from scratch for 20,000 steps to obtain their final test performance.

3) *Mutation and Crossover:* Mutation replaces each genome entry independently with a uniformly sampled class ID from the operator pool with probability $p_{\text{mut}} = 0.10$. Multi-point crossover selects two random cut points in the genome list and exchanges the segment between them between two parent genomes. A *repair* step clamps any out-of-range class IDs to the valid pool after mutation.

F. Overall Pipeline

The complete LIV-TS workflow proceeds in three phases:

- 1) **Search.** Run NSGA-II for 18 generations on the target dataset (ETTh1, $H=96$) using $N_{\text{eval}} = 500$ steps per genome. Record the final Pareto front.
- 2) **Full training.** Retrain the top Pareto-front genomes for 20,000 steps. Report test MSE and MAE.
- 3) **Transfer.** Apply the best genome (by validation MSE) directly to new datasets and horizons without re-running evolution, to assess architectural generalisability.

TABLE III: Architecture search results on ETTh1. Best test MSE genome per (structure, H) reported after 20,000-step retraining. Genome entries are operator class IDs from the pool (Table I and differential variants).

Structure	H	Params	MSE	MAE
S-Mamba	96	528K	0.596	0.547
S-Mamba	336	678K	0.758	0.641
S-Mamba	720	1,590K	0.819	0.670
iTransformer	96	1,188K	0.549	0.502
iTransformer	336	939K	0.809	0.661
iTransformer	720	1,976K	0.808	0.671

TABLE IV: Architecture search results on Exchange-Rate. Best test MSE genome per (structure, H).

Structure	H	Params	MSE	MAE
S-Mamba	96	570K	0.143	0.284
S-Mamba	336	490K	0.770	0.681
S-Mamba	720	1,059K	1.084	0.829
iTransformer	96	1,090K	0.142	0.281
iTransformer	336	1,153K	0.673	0.629
iTransformer	720	1,059K	1.309	0.838

IV. EXPERIMENTS

A. Experimental Setup

Datasets. We evaluate on three benchmarks: ETTh1 [25] (17,420 time-steps, 7 variates, hourly electricity transformer temperature); ETTm1 [25] (69,680 time-steps, 7 variates, 15-minute resolution); and Exchange-Rate [1] (7,588 time-steps, 8 currency exchange rates). For each dataset we report three forecast horizons: $H \in \{96, 336, 720\}$ with a fixed lookback window $L = 96$. Data is split 70/10/20 for train/validation/test and normalised per variate with Reversible Instance Normalisation (RevIN [24]).

Structures. All experiments use two structural templates: S-Mamba (4 genome slots split evenly between variate and temporal backbones) and iTransformer (4 genome slots alternating cross-variate and per-variate sub-layers).

Training. During NSGA-II search, each candidate genome is evaluated for $N_{\text{eval}} = 500$ steps using the Adam optimiser with learning rate 10^{-3} . The top Pareto-front genomes are retrained from scratch for 20,000 steps. All experiments run on a single GPU.

Metrics. We report test MSE and MAE on un-normalised (RevIN-restored) outputs, consistent with the iTransformer benchmark protocol.

B. Experiment 1: Architecture Search Results

We run NSGA-II for 18 generations (population 16) on each (dataset, horizon) pair and retrain the Pareto-front genomes for 20,000 steps. Table III reports the best-MSE candidate per (structure, horizon) for ETTh1, and Table IV for Exchange-Rate. Table V summarises ETTm1 results.

Across all datasets and horizons, the top Pareto-front genomes are consistently *heterogeneous*: no single operator class fills all four slots. For example, the best ETTh1 $H=96$ S-Mamba genome mixes SA-3 (MQA), SA-1 (MHA),

TABLE V: Architecture search results on ETTh1. Best test MSE genome per (structure, H).

Structure	H	Params	MSE	MAE
S-Mamba	96	1,188K	0.503	0.479
S-Mamba	336	563K	0.723	0.604
S-Mamba	720	532K	0.789	0.648
iTransformer	96	1,321K	0.534	0.500
iTransformer	336	467K	0.684	0.592
iTransformer	720	605K	0.826	0.663

TABLE VI: Operator ablation on ETTh1. Test MSE for each single-class baseline and the best NSGA-II heterogeneous genome (MIXED_GA). Parameter counts vary with horizon due to the forecast head size.

Structure	Variant	H = 96		H = 336		H = 720
		Params	MSE	Params	MSE	MSE
S-Mamba	ALL_CONV	582K	0.518	644K	0.700	0.742
	ALL_ATTN	874K	0.548	936K	0.682	0.805
	ALL_FFN	3.2M	0.450	3.3M	0.549	0.657
	ALL_CFC	16.9M	0.493	17.0M	0.705	0.758
	MIXED_GA	578K	0.537	790K	0.680	0.853
iTransformer	ALL_CONV	582K	0.592	644K	0.806	0.809
	ALL_ATTN	874K	0.555	936K	0.663	0.757
	ALL_FFN	3.2M	0.454	3.3M	0.551	0.668
	ALL_CFC	16.9M	0.491	17.0M	0.690	0.746
	MIXED_GA	1.2M	0.464	1.1M	0.677	0.749

SA-3, and SA-4 (GQA) slots—three attention variants with different KV compression ratios—at only 528K parameters. The best iTransformer genome for the same condition combines GConv-2 (long convolution) with differential-FFN and GConv-1 slots. On Exchange-Rate at $H=96$, both structures discover compact (≤ 1.1 M parameter) heterogeneous genomes achieving $\text{MSE} \approx 0.14$, matching the empirical difficulty of short-horizon currency forecasting.

C. Experiment 2: Operator Ablation

To isolate the contribution of operator heterogeneity, we compare five baseline configurations against the best NSGA-II genome (MIXED_GA): **ALL_CONV** (all GConv-2 slots), **ALL_ATTN** (all SA-1 slots), **ALL_FFN** (all GMemless differential slots), **ALL_CFC** (all Cfc differential slots). All variants use the same structure template and are trained for 20,000 steps.

Tables VI and VII reveal several patterns. First, no single operator class dominates across datasets: ALL_FFN achieves the best ETTh1 MSE but uses 3.2M parameters versus 578K–1.2M for MIXED_GA. Second, ALL_CFC incurs 16.9M parameters due to the $16\times$ internal expansion of the Cfc featurizer, making it parameter-inefficient despite competitive accuracy. Third, on Exchange-Rate, MIXED_GA achieves 0.148 MSE (S-Mamba, $H=96$) with 1.6M parameters, matching ALL_ATTN (0.156) at comparable cost and approaching ALL_FFN (0.108) at one-fifth the parameter budget. These results confirm that heterogeneous operator mixtures discovered by NSGA-II provide a better accuracy-compactness trade-off than any homogeneous baseline.

TABLE VII: Operator ablation on Exchange-Rate. Test MSE for single-class baselines vs. MIXED_GA.

Structure	Variant	Params	MSE ($H=96$)	MSE ($H=336$)
S-Mamba	ALL_CONV	582K	0.672	1.372
	ALL_ATTN	874K	0.156	0.618
	ALL_FFN	3.2M	0.108	0.625
	ALL_CFC	16.9M	0.227	1.090
	MIXED_GA	1.6M	0.148	0.726
iTransformer	ALL_CONV	582K	0.489	1.457
	ALL_ATTN	874K	0.203	0.763
	ALL_FFN	3.2M	0.099	0.617
	ALL_CFC	16.9M	0.393	1.289
	MIXED_GA	1.0M	0.158	0.795

TABLE VIII: TiDAR inference speed-accuracy trade-off on Exchange-Rate $H=96$, S-Mamba. Speedup is relative to full-AR ($k=96$). MSE is on the test set.

Mode	k	Speedup	Latency (ms)	MSE
Draft	0	96.9$\times$	0.152	0.169
Partial-AR	1	48.5 $\times$	0.304	0.169
Partial-AR	4	19.4 $\times$	0.760	0.169
Partial-AR	16	5.7 $\times$	2.586	0.167
Full-AR	96	1.0 $\times$	14,754	0.241

Experiment 3: TiDAR Inference Speedup

We evaluate TiDAR’s inference trade-off between speed and accuracy on Exchange-Rate ($H=96$) using the best S-Mamba genome from Experiment 1. Five operating modes are compared: pure draft ($k=0$), partial-AR with $k \in \{1, 4, 16\}$ autoregressive refinement steps, and full-AR ($k=H=96$). Latency is measured as median milliseconds per sample over 86 batches.

Table IX shows that the draft mode achieves a **96.9 $\times$** wall-clock speedup while raising MSE by only 0.000 (0.169 vs. 0.169 for $k=1$) relative to partial-AR refinement. Strikingly, the full-AR mode ($k=96$) produces *higher* MSE (0.241) than draft or partial-AR modes, indicating that the TiDAR parallel training objective teaches the model to forecast more reliably in non-sequential mode than in the sequential mode it was not primarily trained for. Even a single AR refinement step ($k=1$, 48.5 $\times$ speedup) matches draft quality. A comparable evaluation on ETTh1 $H=96$ yields an 80.8 $\times$ draft speedup (MSE 0.640 vs. full-AR 0.776), confirming the pattern across datasets.

E. Experiment 4: Cross-Dataset Architecture Transfer

We test whether genomes discovered on one dataset transfer without re-running evolutionary search. Two source genomes are evaluated: the best $H=96$ genome found on ETTh1 (Experiment 1, S-Mamba), and the best $H=96$ genome found on Exchange-Rate. Each genome is trained from scratch on the target dataset and evaluated at matching horizons. The baseline is an all-Mamba (Rec-1) genome of comparable depth.

Table X shows that the ETTh1 genome transfers successfully to ETTh2 (+31% at $H=96$ and 336) and Exchange-Rate (+63% at $H=96$), where dataset characteristics are compatible—hourly multivariate electricity and smooth currency dy-

TABLE IX: Transfer from ETTh1 (S-Mamba, H=96 search genome). Test MSE of the transferred genome vs. the all-Mamba baseline on four target datasets. Positive Δ means baseline is worse (transfer helps).

Target	H	Searched	Baseline	$\Delta\%$
ETTh2	96	0.444	0.649	+31.6%
	336	0.735	1.071	+31.3%
	720	2.173	1.365	-59.2%
ETTh1	96	0.601	0.551	-9.2%
	336	0.781	0.688	-13.5%
	720	0.779	0.738	-5.7%
ETTh2	96	0.273	0.251	-8.9%
	336	0.618	0.717	+13.8%
	720	0.769	1.210	+36.5%
Exchange	96	0.147	0.405	+63.6%
	336	0.718	0.825	+13.0%
	720	1.301	1.482	+12.2%

TABLE X: Transfer from Exchange-Rate (S-Mamba, H=96 search genome). Test MSE vs. all-Mamba baseline. Positive Δ means transfer helps.

Target	H	Searched	Baseline	$\Delta\%$
ETTh1	96	0.646	0.639	-1.2%
ETTh2	96	0.744	0.580	-28.1%
ETTh1	96	0.547	0.545	-0.3%
ETTh2	96	0.346	0.260	-32.8%

namics, respectively. Transfer to ETTm (15-minute resolution) is mixed: the ETTh1 genome slightly underperforms the all-Mamba baseline at H=96 and 336 but recovers at H=720.

Table XI shows that the Exchange-Rate genome does not transfer well: it matches the baseline on ETTh1 and ETTm1 (within 1%) but underperforms by 28–33% on ETTh2 and ETTm2. This asymmetry suggests that the structural inductive bias discovered for currency rates (dominated by smooth trends and low noise) is too specialised for the richer multivariate dynamics of the ETT benchmarks.

F. Experiment 5: TiDAR α Sensitivity

We ablate the balance coefficient α in the TiDAR loss (Eq. (13)) on the best S-Mamba genome from Experiment 1 (ETTh1) across four horizons. $\alpha = 0$ recovers pure parallel prediction; larger α increases the weight of the AR auxiliary task.

Table XII reveals a consistent trend: moderate AR regularisation ($\alpha \in [0.5, 2.0]$) outperforms both extremes across all horizons. Pure parallel prediction ($\alpha = 0$) performs worst, indicating that the AR auxiliary task provides useful structural supervision even when inference is performed in draft mode. The optimal α shifts with horizon: $\alpha = 2.0$ is best for H=96 and H=192, while $\alpha = 0.5$ is best for H=336 and $\alpha = 5.0$ for H=720. This horizon-dependent optimum is consistent with the idea that longer forecasting horizons benefit more from sequential structure, making stronger AR regularisation preferable.

TABLE XI: Test MSE for varying α on ETTh1 (S-Mamba). Genome fixed to the best H=96 candidate from Experiment 1. Bold indicates best MSE per horizon.

α	H = 96	H = 192	H = 336	H = 720
0.0	0.756	0.835	0.919	1.101
0.1	0.704	0.825	1.006	1.097
0.5	0.694	0.827	0.852	1.087
1.0	0.729	0.747	0.955	1.071
2.0	0.671	0.739	0.947	1.063
5.0	0.726	0.840	0.894	1.009

REFERENCES

- [1] H. Wu, J. Xu, J. Wang, and M. Long, “Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 22 419–22 430.
- [2] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, “Informer: Beyond efficient transformer for long sequence time-series forecasting,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [3] Y. Liu, T. Hu, H. Zhang, L. Wu, S. Wang, L. Ma, and M. Long, “itransformer: Inverted transformers are effective for time series forecasting,” in *International Conference on Learning Representations*, 2024.
- [4] W. Wang, Y. Ma, H. Ma, and L. Sun, “S-mamba: Inter-series dependency modeling with state space models for multivariate time series forecasting,” *arXiv preprint arXiv:2403.11144*, 2024.
- [5] T. Elsken, J. H. Metzen, and F. Hutter, “Neural architecture search: A survey,” *Journal of Machine Learning Research*, vol. 20, no. 55, pp. 1–21, 2019.
- [6] B. Zoph and Q. V. Le, “Neural architecture search with reinforcement learning,” in *International Conference on Learning Representations*, 2017.
- [7] M. Poli, S. Massaroli, E. Nguyen, D. Y. Fu, T. Dao, S. Baccus, Y. Bengio, S. Ermon, and C. Ré, “Hyena hierarchy: Towards larger convolutional language models,” in *International Conference on Machine Learning*, 2023.
- [8] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [9] Anonymous, “Think in diffusion, talk in autoregression: Parallel multi-step prediction for time series,” *arXiv preprint*, 2024, tiDAR framework.
- [10] R. Hasani, M. Lechner, T.-H. Wang, M. Chahine, A. Amini, and D. Rus, “Liquid structural state-space models,” *arXiv preprint arXiv:2209.12951*, 2022.
- [11] A. Zeng, M. Chen, L. Zhang, and Q. Xu, “Are transformers effective for time series forecasting?” in *AAAI Conference on Artificial Intelligence*, 2023.
- [12] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” in *International Conference on Learning Representations*, 2023.
- [13] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, “Timesnet: Temporal 2d-variation modeling for general time series analysis,” in *International Conference on Learning Representations*, 2023.
- [14] A. W. Thomas, R. Parnichkun, A. Amini, S. Massaroli, and M. Poli, “STAR: Synthesis of tailored architectures,” *arXiv preprint arXiv:2411.17800*, 2024.
- [15] S. Massaroli and M. Poli, “Linear input-varying systems for sequence modeling,” *arXiv preprint*, 2024, IIV operator theory underlying STAR.
- [16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [17] R. Hasani, M. Lechner, A. Amini, L. Liebenwein, A. Ray, M. Tschaikowski, G. Tanner, and D. Rus, “Closed-form continuous-time neural networks,” *Nature Machine Intelligence*, vol. 4, pp. 992–1003, 2022.
- [18] N. Shazeer, “GLU variants improve transformer,” *arXiv preprint arXiv:2002.05202*, 2020.
- [19] A. Gu, K. Goel, and C. Ré, “Efficiently modeling long sequences with structured state spaces,” *arXiv preprint arXiv:2111.00396*, 2021.
- [20] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.

- [21] J. Gu, J. Bradbury, C. Xiong, V. O. Li, and R. Socher, “Non-autoregressive neural machine translation,” in *International Conference on Learning Representations*, 2018.
- [22] N. Shazeer, “Fast transformer decoding: One write-head is all you need,” *arXiv preprint arXiv:1911.02150*, 2019.
- [23] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zeiler, S. Sanghai, and Y. Tay, “GQA: Training generalized multi-query transformer models from multi-head checkpoints,” *arXiv preprint arXiv:2305.13245*, 2023.
- [24] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, “Reversible instance normalization for accurate time-series forecasting against distribution shift,” in *International Conference on Learning Representations*, 2022.
- [25] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, “Informer: Beyond efficient transformer for long sequence time-series forecasting,” in *AAAI Conference on Artificial Intelligence*, 2021, introduces the ETT benchmark datasets.